

大規模言語モデルによる並行性制御の空間外合成 —有限フラグ空間での否定的結果と、 正しさゲートを毎反復回す合成ループ—

(著者名を記入)¹

概要: トランザクション処理系の並行性制御 (CC) の最良の形はワークロードによって変わる。本稿は、対象実装が取りうる動作の集合そのものを大規模言語モデル (LLM) にコードとして拡張させ、生成のたびに直列化可能性を検査しながら、ワークロードごとに検証済み (certified) の選択を返すシステム Izanagi を報告する。ここで certified とは、指定した検証構成で観測した YCSB の point read / write の trace 上で判定器が直列化可能と判定したことであり、性能の認証ではない。Silo の有限フラグ空間では LLM 誘導探索は貪欲探索を上回らず、欺瞞的な負荷では有意に有害だった。一方、フラグ空間の外へ合成した「abort 後の待機量の静的固定」は、旧環境の記述的な sweep で無 backoff 対照に対し write-heavy +38.3%, balanced +11.3%, read-heavy -6.6% を示した。別の合成軸である abort 要因別の gate が既知軸の最良を超えるという主張は、結果既知の事前登録付き追試として測って不成立だった。限定したスコープで正しさゲートを一度も緩めずに合成を certified まで回すループは成立したが、ワークロード条件付き合成の因果的な証拠と LLM の必要性は未取得である。

1. はじめに

トランザクション処理系の並行性制御 (concurrency control, CC) は、競合をいつ検出し、abort の後にどれだけ待ち、書き込み対象のロックをどの順で取るか、といった判断の集まりである。その最良の形はワークロード — 読み書きの比率、アクセスの偏り、スレッド数 — によって変わる。ワークロード特化を機械に任せる既存の取り組み (設定パラメータの調整、あらかじめ定義した操作集合の中での方策探索、CC を学習可能な関数として近似する方法) は、対象実装が取りうる動作の集合 (action vocabulary) を固定したまま、その中から選ぶ。本稿が扱う問いはその一段外にある。**対象実装の動作の集合そのものを、コードとして拡張できるか。**すなわち、大規模言語モデル (LLM) に CC 実装の中の新しい動作を生成させ、生成のたびに直列化可能性を検査しながら、ワークロードごとに検証済み (certified) の選択を出せるか、である。

この問いには安全性の問題が付随する。生成された変異は正しさを壊しうるし、性能を最適化する圧力は正しさの検査を緩める方向に働く (評価器の穴を突いた変異が「速い」と判定される、いわゆる reward hacking)。したがって正しさの検査は、探索の最後にまとめて回す関門ではな

く、毎反復で回して次の生成の入力にする必要がある。

本稿は、この問いに取り組むシステム Izanagi を報告する。Izanagi は CCBench [1] の既存実装を素材とし、(i) certified な選択結果、(ii) なぜそれが速いかの説明、(iii) 試行錯誤の再現可能な記録、を返す。**証拠が支持すれば新しい変異を、支持しなければ既定実装 (stock) または「差なし」を返す。新しい CC が必ず出てくる契約ではない。**先行する取り組みとの差は、(1) 対象がトランザクションの並行性制御であること、(2) 対象実装の action vocabulary そのものを拡張すること、(3) 正しさゲートを毎反復回すこと、の3点で書く。3点はいずれも実装と計測が動いている面であるが、3点が一続きの科学的な証拠の鎖を成すことは本稿では実証していない。

位置づけについて本稿が言えるのは次の1文までである — 「本調査では、LLM が、トランザクションの並行性制御を対象として、アクション空間自体をコードで拡張する (固定した原始操作語彙の中で生成するのではなく) 既存例を発見していない。」この記述は3条件 (LLM が / トランザクションの並行性制御を対象として / アクション空間自体をコードで拡張する) のすべてに限定され、どれ1つを落とした短縮形では成り立たない。先行研究の調査は限定付きで止めており、完了でも優先権の主張でもない。この1文の根拠となった調査の後に確認した関連研究があり、1

¹ (所属を記入)

文との関係は判定していない (2 節).

本稿の結果は次の 4 つである. 各項の角括弧は証拠の種類を示す.

- (1) **有限フラグ空間での否定的結果** [実証済み]. Silo の既存フラグ空間では, LLM 誘導探索が LLM を使わない貪欲探索より少ない評価回数で最良群に到達するという優越は示されず, 欺瞞的な構造を持つ負荷では誘導側の到達コストが有意に高かった (4.1 節).
- (2) **空間外合成の一事例** [実証済み, 旧測定環境の記述的な一事例]. 評価指標の帰属を起点に, 対象実装のフラグ空間の外への軸「abort 後の待機量の静的固定」を合成し, 3 ワークロードのうち 2 つで勝ち 1 つで負けた. backoff という機構そのものは既知であり (2 節), 本稿が示すのは機構の新しさではなく, 合成が対象実装の空間の外へ到達した一事例である. 現行環境での正式な対測定は同じ向きの符号を示したが, これは記述的な照合であって再現の判定ではない (4.2~4.4 節).
- (3) **事前登録した主張の失敗報告** [測って届かなかった]. 別の合成軸 (abort 要因別の gate) が既知軸の最良を超えるという主張は, 結果既知の事前登録付き追試として測って不成立だった (4.5 節).
- (4) **正しさゲートを緩めない合成ループの成立** [運用上の証拠] と, 最終候補の長時間の正しさ検証 [固定条件で certified な正しさ] (4.6~4.7 節).

結果が支持するのは, 価値がフラグ空間の探索ではなく空間の外への合成にあるという対比である. 一方, 合成の一般性, ワークロード記述子を条件にした生成の因果的な証拠, LLM の必要性 (条件付き優越の対照), 正しさシグナルの還流による改善は, いずれも未取得である (6 節).

以下, 2 節で関連研究, 3 節で Izanagi の設計と評価の方法, 4 節で評価, 5 節で考察, 6 節で限界を述べ, 7 節でまとめる.

2. 関連研究

本研究が素材とする CCBench [1] は, 主要な in-memory CC プロトコルを共通基盤に再実装し同一ワークロードで比較する分析基盤である. CC を設計・学習する系譜として, Polyjuice [2] は CC をアクションに分解し進化的に学習する. 同論文は Silo 上の backoff も学習し, TPC-E の高競合下での利得は主に学習した backoff によると報告している. したがって本稿の静的 backoff の利得は backoff という機構の新しさを示すものではない. 本稿が示すのは, 対象実装 (CCBench の Silo) のフラグ空間の外へ合成が到達した一事例である. CCaaLF はこれを関数として一般化し NeurCC [3] に改名され, ATCC [4] は未知ワークロードへの適応を扱う. Izanagi はこの問題設定 (CC をアクション列に分解して探索する) を借りるが, 配合探索や関数近似ではなく, アクション空間自体を LLM がコードで拡張す

る点で異なる. 学習型 CC は公開実装の基盤が CCBench と異なるため定量比較は行わない. AI を用いない例として, Declarative Concurrent Data Structures [5] は宣言的 DSL の仕様からコンパイラが同期を注入し, YCSB で最大 2 倍の性能を報告する. この研究とは対象 (トランザクションの CC) を共有するが, LLM も進化探索も用いずコンパイラのパスで自動化する点, 注入するプロトコルが 1 種で空間を拡張しない点, 正しさ検証器をループ内に持たない点で異なる.

DBMS の設定 knob を自動チューニングする系譜は, OtterTune [6]・CDBTune [7]・QTune [8]・UDO [9] の機械学習・強化学習ベースの枝と, DB-BERT [10]・GPTuner [11]・SysInsight [12] の LLM ベースの枝からなる. この系譜の介入面は設計者が事前に用意した knob の次元に閉じ, 探索空間が最大の UDO ですら人間が用意した有限候補からの選択である. SysInsight は DBMS のソースコードを静的解析と LLM で読み, knob が支配する関数の因果経路から調整仮説を立てるが, 介入の実体は既存 knob 上の条件付き方策であり, 並行性制御の分岐構造や新しい待機戦略を追加しない. Izanagi はワークロード特化の DB 自動最適化という目標を, knob 空間からコード空間へ持ち出す位置に立つ.

DB の中核部品を学習物で置き換える系譜として, learned index (Kraska ら [13], 更新対応の ALEX [14], 最悪ケース保証を証明する PGM-index [15]) と learned query optimizer (Neo [16], 既存 optimizer を粗粒度ヒントで操縦する Bao [17]) がある. この系譜は実行経路にモデル推論を挟む model-in-the-path だが, Izanagi は学習費用を合成時に払い実行時は推論を挟まない code-as-output であり, トランザクションごとに μs 以下の critical path を持つ CC ではこの違いは必然である.

LLM を変異オペレータにした進化探索でコードを合成する系譜として, FunSearch [18] は LLM と evaluator を対に best-shot prompting で母集団を保ち, AlphaEvolve [19] は評価スコア付きの勝ちプログラムを prompt に注入し diff を生成し, ShinkaEvolve [20] は同一問題設定を解く公開実装である. Izanagi は編集面を機械的に限定する仕組み (EVOLVE-BLOCK と diff 形式) や評価カスケードの発想を採るが, これらがサンプル効率の源泉とする「勝ちプログラムを生成側に見せる」機構は採らず, 直列化可能性を毎反復検査して正しさを破る変異を即座に棄却する. LLM の進化探索でトランザクションのスケジューリング方策を設計した研究 [21] もあるが, 評価は Python シミュレータで行われ, 直列化可能性の検査についての記述は無い.

LLM エージェントがワークロード仕様からシステムを合成する系譜に, Jitskit [22] (KV ストア), VibeServe [23] (LLM serving), IDS [24] (コードと証明を同時に育てる検

証付き合成)がある。Izanagi は Jitskit から spec cards・reward hack カタログ・leading indicators・敵対的監査役・却下設計の記憶を借りるが、これらはいずれも並行性制御を対象にしない。並行プログラムの検証付き合成の例として、CIR+CVN [25] は LLM に自然言語仕様から同期構造を書かせ Petri ネットへ機械翻訳し反例を差し戻すが、対象はミューテックスや条件変数のデッドロックとシグナル消失で、並行性制御や性能は扱わず、原始操作の語彙も形式系の閉じた集合で拡張されない。検証を通った編集だけ採用して改善する自己改善の系譜 (SkillOpt [26], Self-Harness [27]) は、正しさゲートを緩める変異を採らない規律の外部裏付けとして参照するが、自己改変ループの機構自体は実装しない。

以上の差別化は 1 節に記した 1 文の範囲に限られ、先行研究に対する優越や網羅を主張するものではない。先行研究の調査は費用対効果を理由に限定付きで止まっており、対象の空白についての検索は事前登録した索引の一部で実行されたに留まり、説明可能性の観点からの検索は行っていない。前段のトランザクションのスケジューリング方策の研究 [21] は、この 1 文の根拠となった調査の後に確認したもので、3 条件に当たるかは判定していない。本節が挙げなかった先行研究の存在を否定するものではなく、調査の停止を完了や優先権の主張の根拠として用いない。

3. Izanagi の設計と評価の方法

3.1 構成と出力契約

Izanagi は CCBench の既存実装を素材とし、既存フラグの組合せを探索する操作と、指定したソース領域へコードを挿入して実装の振る舞いを変える操作を組み合わせる。列挙できる軸内の探索は機械探索に渡し、LLM は変更の方向やコード片を提案する。現在扱う軸は、abort 後の待機量を定める一文を生成する backoff 軸、書き込み集合内のロック取得順序を決める sort 軸、abort の要因ごとに待機の発火可否を決める trigger-gating 軸の 3 つであり、各軸は固有の文法と制約の下で閉じている。探索と合成が実際に回ったのは、Silo の 1 プロトコルとこの 3 軸だけである。これらの実装から、任意の CC アルゴリズムを自動合成できるとは結論しない。合成対象は当初 Silo の 1 プロトコルに限定されていたが、2026-09-17 にこの限定を解いた。第 2 の対象として検討中の MoCC については、編集領域とする温度述語の骨格 template と、その接続を機械検査する準備段階の実証を行ったが、これは探索の解禁でも正式な軸の採用でもなく、非 Silo の性能比較は本稿の評価には含まれない。

基本の評価順序は、提案、差分検査、検証用・性能用の二つの別ビルド、実行トレースによる検証、独立した性能測定、記録と次の提案へのフィードバックである。出力契約は、証拠が支持すれば certified な新しい候補を、支持し

なければ既定実装 (stock) か、差を支持できない同率 (tie) かを区別して正直に返すことであり、候補に性能値が無い場合や比較の前提が成立しない場合を有効な tie と取り違えない。新しい CC が必ず出てくる契約ではない。

出力契約が含む「説明」は、提案・拒否・検証・測定の出所を追う試行記録の射影であり、critic の帰属記録を非 certifying な二次的な view として示す機序仮説層にとどまる。LLM が述べた機序は仮説として扱い、説明の生成や記録の整合性から、説明の忠実性や因果機序の同定を推論しない。

3.2 変異の生成と編集範囲

提案生成は planner と coder に分ける。planner は現在の性能指標と反復履歴から変更方向を提案し、coder はその方向と与えられた基準情報から具体値とコード片を返す。critic は評価後の指標や拒否理由を読み、次の方向を提案する。

編集領域は EVOLVE-BLOCK の開始・終了マーカーで指定する。テンプレートは合成側の分岐と既存実装を保持する分岐を両方用意し、coder が変更してよいのは合成側の内部だけである。backoff 軸はコードを出力する形式であっても、この軸で探索する自由度は abort 後の待機量を定める数値一つに絞られる。検査は、マーカー・分岐骨格・既存実装側の保存と編集範囲の確認に加え、ホストへの作用を検査する有限の字句規則と軸固有の文法を適用し、backoff 軸では提案値とコード中の数値リテラルの一致も確認して、記録する候補と実際にビルドする実装の対応を保つ。字句検査だけで任意コードの意味的安全性が証明されるとは扱わない。

ビルドの識別には、フラグの構成に加え、前処理後のソースに由来する識別情報 (src.token) を用いて、記録する候補と実際にビルドされた実装の対応を保つ。ただし #define / #undef 等の前処理指令と include の相対位置、push_macro / pop_macro の扱いは識別の限界として残る。同じ genome を持つ既評価候補の重複提案では、保存済み証拠を確認して評価を再利用する場合があるが、検査通過だけの試行やビルドを省いた配線確認は、検証済み候補へ昇格させない。検査に通らない提案はビルドへ進めず、拒否の種類・理由・対象領域を記録し、次の提案の入力にする。

3.3 正しさ検証

正しさは、トレースを有効にした別ビルドを実行して検証する。判定器は実行トレースから読み書きの依存関係 (書き込み間・書き込みから読み込み・読み込みから後続書き込み) に基づく直列化グラフを構築し、反依存を含む循環 (G2) までは検出する。anomaly を検出した候補はその時点で失格とし、性能が高くても採用しない。空トレース・解析不能・

実行失敗も正しさを確かめられた成功例には数えない。

判定器の検出力は、2026 年 6 月の判定器で双方向に確かめた。読み取り検証を意図的に外して壊した Silo で 1,310 件、無改変の snapshot isolation で 3,576 件の G2 を検出して非直列化可能と判定し、素の Silo と既知の 8 構成は直列化可能と判定した。ただし snapshot isolation の件数は当時の trace 形式を受理した判定器での値であり、現行の判定器はその形式を受理しないので、現行の判定器では再現していない。判定器自身への敵対的な検証は、不正な trace で赤を隠せる穴を見つけたので、判定を 3 値 (直列化可能 / 非直列化可能 / 判定不能) にして安全側に倒した。

本稿で certified と書くときは、指定した検証構成 (workload・スレッド数・実行時間・反復数) で取得した観測トレースについて、判定器の意味論 (YCSB の point read/write に限った観測トレース上の直列化可能性、反依存を含む循環 G2 までの検出) に照らして anomaly が検出されず受理されたことに限る。これは実際にビルドされた bytes についての判定であり、要求した構成がビルドされたことを含意せず、性能の判定でもなく、プログラムの全入力・全実行順序に対する形式証明や論文での性能優越の成立を意味しない。

性能の比較とは別に、正しさ側の追加検証を 2 つ行った。(i) 採用した静的 backoff の候補 2 つについて、現行ソースのトレース有効ビルドで 3 workload の検証を繰り返す検証相、(ii) 合成軸 (abort 要因別の gate) の最終候補について、独立プロセスがそれぞれ自己シードする長時間・複数反復の検証である。(ii) の判定は、判定集合 (本走 24 枠と、校正で判定器が完走した検証) に anomaly が 1 件以上あるか、判定が直列化可能以外 (判定不能を含む) の検証が 1 件でもあれば直ちに失格 (再実行しない)、失格でなく本走のすべてが基準を満たせば pass、どちらでもなければ未確定とする 3 値の規則で、結果を見る前に固定した。(i) と (ii) は対象と長さが異なる別の検証であり、結果を比較しない。(ii) の「種を変えた」は独立プロセスの自己シード、「長時間」は実行時間 10 秒という操作的定義であり、乱数列の独立性そのものは検証していない。いずれも性能値を含まない (結果は 4.6 節)。

判定器は、対象プロトコルの証明面 (ロック被覆等の観測点) がソースに実在するかも検査し、証明面を持たないプロトコルのトレースは certified を名乗れない。検証構成には、既定の小規模な構成 (以下 legacy 構成) と、性能を測った条件に合わせた構成 (以下 性能条件) があり、追加構成を指定した場合はその全構成・全反復の合格を要求する。性能を測ったワークロードそのもので正しさを求める経路では、legacy 構成 1 回と性能条件 5 回の検証を要求する。ある設定での合格を、試していない負荷や別プロトコルの正しさへ一般化しない。

拒否は単なる二値の失敗にせず、判定器が返した構造化

診断 (どの取引間のどの依存で循環ができたか) を試行記録へ残す。依存関係上の反例、生存性の失敗 (コミットが得られない等)、差分検査の拒否を区別し、どの構成で失敗したかを次の提案へ渡す。

3.4 性能測定との分離と判定の事前登録

性能は、トレース処理を実行時分岐ではなくコンパイル時に完全に除去した別ビルドを、別に実行して測定する。候補と比較対象はともにトレース無効でそろえ、検証のための観測処理を測定対象の CC に混ぜない。

測定条件は、レコード数・スレッド数・ワークロード・実行時間・反復数・環境契約で指定し、探索規模には較正によって必要な競合やキャッシュ挙動を再現する最小規模を用いる。測定層は実行の競合を確認し、スループットの反復標本・中央値・変動係数・abort 率等を記録する。規定範囲の再測定でも安定しない場合は unstable として残し、測定が完了したこと、正しさを通過したこと、安定した比較材料になったことを別々に判定する。以下、protocol は事前登録した測定と判定の手順 (CC プロトコルとは別)、campaign は同じ設定の走行をまとめた実行単位、attempt は事前登録した実験を投入した 1 回、arm は比較する構成 (採用側と無 backoff 側など)、cell はワークロードと構成の組ごとの測定の単位、cohort は同じ手順で独立に行った 1 回分の測定群を指す。性能比較は同一 campaign 内の対測定で行い、凍結した過去値を比較の基礎にしない。一回の測定内のばらつきと、別 run 間の比較に使う noise floor (床値) は区別し、比較対象と競合条件に対応する床値を用いて、別条件で得た定数を無条件に流用しない。

ビルドに要求した条件 (定義マクロ) が実際に供給され、実行側で同じ意味を持つかは、測定前に別の関門で検査し、供給の有無と意味の一致を独立の必須項目として扱う。

事前登録した対比較では、比較対象・検証構成・床値・標本設計・報告規則を結果を見る前に固定し、登録した区分 (床の外か内か、未解決か) で判定して向きは平均の符号で読む。試行数や検証相を満たさない中間走行は記述的に報告するにとどめ、有意差や LLM 固有の優越を主張しない。

3.5 評価環境と記録の扱い

評価環境は、過去に使用した linux-baremetal 環境と、現行の Pegasus 計算ノード環境を区別する。全実行ログに環境タグを必須項目として持たせ、性能比較は同じ環境タグの実機数値だけで行い、異なる環境で得た値を無条件に混ぜない。主要な動作点は 48 スレッド・YCSB 系 3 workload (balanced・read-heavy・write-heavy) である。

事前登録した対比較の非認証 lane では、2 回の attempt が完走した (各 attempt は独立の観測であり、プールしない)。結果は 4 節に譲る。

素材コーパスの版 (pin) の前進後、旧系列の実験を現行版の main から再開するための整合を系列ごとに実測したところ、通る系列と、起動時の検査だけが塞ぐ系列とに分かれた。その後、旧系列の再開は本研究の必須経路から外した。本稿の測定はそれぞれ記録された当時の版で得たものであり、版の前進はそれらの記録を無効にしない。

4. 評価

本節は、4.1~4.4 節で有限フラグ空間での探索の比較と静的 backoff の測定を、4.5~4.9 節で既知軸最良との直接比較・正しさの追加検証・合成ループ・機序の帯域外・別プロトコルの観測を述べる。各小節の走行は protocol・事前登録・環境・attempt のいずれかが異なる互いに独立した走行であり、小節をまたいで値をプールせず、差や比も作らない。observed-positive などの status は protocol の出力であって研究の成否のラベルではない。

4.1 有限フラグ空間での誘導探索の比較

まず、既存の設定を選ぶ操作に LLM を用いる価値を調べた。Silo の 8 構成からなる有限フラグ空間を対象に、LLM 誘導と、LLM を使わない貪欲探索を比較した。評価は既存の測定値を再生する方式で行い、試行数は新しい性能測定の反復数ではない。指標は最良群に到達するまでの評価回数 (少ないほどよい) で、未到達は予算上限の 8 回として数えた。誘導側には評価済み構成の指標だけを示し、最適解の明示的な事前知識を除いた専用の critic を用いた。この比較はコード生成の比較ではない。

結果を表 1 に示す。write-heavy では誘導側のコストが高く、この判定は 2 ワークロード × 3 比較の Holm 補正後も維持された。未到達の 8 試行は、最良群へ到達しないまま停止した試行 (誤収束) である。balanced で有意差が出なかったことは両者の等価性の証明ではなく、最良群が 8 構成中 4 構成を占める read-heavy は識別力のある比較として扱わない。少なくともこの有限空間では、指標を言語的に解釈することが貪欲探索を超える探索効率をもたらしたとはいえない。この否定的結果を、LLM によるコード生成一般や、4.2 節以降の操作を拡張する合成へ一般化しない。

4.2 静的 backoff の sweep (旧環境)

abort 後の待機量を静的に指定する操作を Silo へ追加した。既存フラグ空間の外へ操作の集合を広げた事例であり、backoff という機構自体は既知である (2 節)。待機量は機械的な sweep で探索した (図 1)。旧 linux-baremetal 環境で、静的 6 点 (2, 5, 10, 25, 50, 100 μ s) のうち中央値が最大の点は、同じ sweep 内の無 backoff 対照 (BACK_OFF=0) の中央値に対し、write-heavy (10 μ s) で +38.3%, balanced (5 μ s) で +11.3%, read-heavy (2 μ s) で -6.6% だった。

read-heavy では 6 点すべてが対照を下回った。分母はこの対照 1 本であり、CCBench 既定の適応 backoff ではない。

この 3 値は旧測定契約下の記述的な一事例であり、事前登録も有意差の判定も区間推定も持たない。性能は perf stat の下で取得しており、絶対スループットを headline 値にしない。採用点を単一の処理の一般効果として平均せず、現行環境の値とも前後比較・プールしない。正しさは当時の判定器による別走行で 24 件すべて anomaly 0 だったが、検証の引数やトレースは記録に無く、3 値の正しさは「backoff は正しさに影響しない」という機序論証による外挿である。3 値は別環境・別 CCBench 版・別測定契約の記述の結果であり、現行環境の別 attempt での certification (4.3 節) はこの 3 値へ遡らない。

機序について述べるのは sweet-spot 帯 (0~10 μ s) までである。write-heavy の機序診断 (perf record 下の別の profile 系列で、図の射程外) では、abort 率が 0 μ s の 0.815 から 10 μ s の 0.494 へ約 4 割減り、有用 IPC はほぼ一定で、total IPC の低下は純粋な spin の希釈だった。25~100 μ s では有用 IPC 自体が下がる。

4.3 現行環境の正式な対測定

測定当時の現行環境 (Pegasus) で、採用した静的 backoff を無 backoff 対照 (BACK_OFF=0) と対にして測った正式な走行が 3 つある (表 2 上段)。3 つは protocol・事前登録・attempt が異なる独立した走行で、1 つの横断実験ではない。第 1 の走行 (write-heavy と balanced) の observed-positive は、「採用側 cell の中央値スループットが同じワークロードの stock 側 cell (無 backoff) を上回るか」を 2 ワークロードの論理積で問うた protocol の status であり、ワークロードごとの判定ではない。第 2 の走行 (read-heavy) の status は reject だった。第 3 の走行は balanced の別の事前登録で、結果を見る前に 2 点と中央値比の規則を固定し、旧環境の +11.3% を期待値にしていない。その初回の解析は判定側の述語が拒否し、受理集合を広げる変更を含む是正の後に同じ保全成果物で accepted を得た (事前登録と規則は不変)。どの効果量にも信頼区間も有意差判定も無く、accepted も有意差の判定ではない。

符号は write-heavy・balanced で正、read-heavy で負で、旧環境の 3 値と 3 ワークロードとも一致したが、これは記述的な照合であって再現の判定ではない。環境・CCBench の版・測定契約が異なるので旧値とプールせず、改善幅の増大・環境効果・再現性の成立も導かない。read-heavy の reject も他の read 比率・機体・版へ外挿しない。

正しさはトレース有効の別ビルド・別走行から来る。第 1・第 2 の走行の 6 cell は legacy 構成 1 回と性能条件 5 回の検査がすべて、第 3 の走行の 2 arm は legacy 構成で各 1 回、certified である。ここでの certified は YCSB の

表 1 有限フラグ空間 (既存 8 構成) の探索コスト. 平均は最良群に到達するまでの評価回数 (未到達は予算上限の 8 回). A は同数を半分に数えた「誘導のコストが食欲より小さい」確率優越で, 0.5 が同水準に当たる. 試行は既存の測定値の再生であり, 新しい性能測定
の反復ではない.

負荷	誘導の試行数	誘導の平均	食欲の平均 (500 試行)	誘導の未到達	A	読み取れる結果
balanced	12	3.750	4.234	0/12	0.5813	有意差を示さない ($p \approx 0.16$)
write-heavy	12	6.333	4.362	8/12	0.2304	誘導側のコストが高い (片側 exact permutation $p = 0.0002521$)
read-heavy	6	1.667	1.756	0/6	—	最良群が 8 構成中 4 構成を占め, 識別力のある比較として扱わない

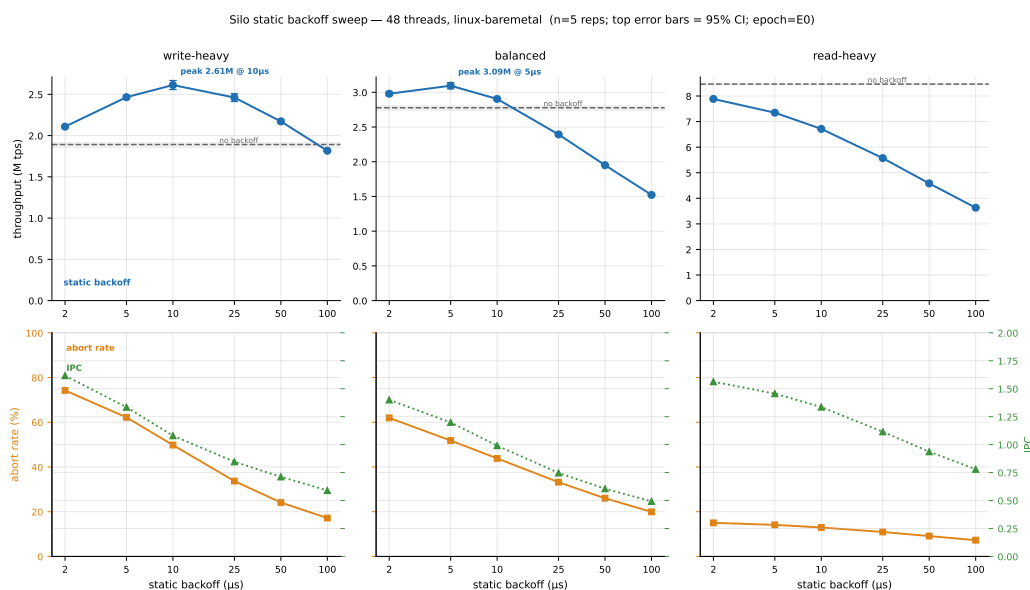


図 1 旧 linux-baremetal 環境における Silo の静的 backoff sweep (3 ワークロード). 上段は各設定 5 反復の標本平均スループットと t 分布による 95% 信頼区間. 灰色の破線と帯は同じ sweep 内の無 backoff 対照 (`BACK_OFF=0`, `BACKOFF_FIXED=-1`) の標本平均と 95% 信頼区間で, これが利得の分母であり, 図中の基準線はこの 1 本だけである. 下段は abort 率と IPC で, 集約 1 点のため信頼区間を付けない. 測定条件は 3 ワークロード共通 (レコード数 1,000,000, 48 スレッド, Zipf 0.9, read-modify-write 無効, 各実行 3 秒, `clocks_per_us = 1,800 TSC tick/μs`, トレース無効, `numactl --interleave=all`) で, 記録された実行フラグの上では read 比率 (5%, 50%, 95%) だけが異なるが, 測定日は同一ではない. 縦軸はパネルごとに独立で, パネル間で線の高さや傾きを比べてはならない. データは判定器の同一性を束縛する仕組みがまだ無かった時点の記録の読み出しであり, 現行の対測定契約より前の記述的結果である. 本図は論文の利得率の出所ではない. 論文値 (+38.3%, +11.3%, -6.6%) は各側の中央値の比で, 図の点推定は標本平均である (平均からは +38.1%, +11.4%, -6.9%). 両者は同じ生値の別の要約で, 丸め違いではない. 絶対スループットは headline 値の出所にしない. spin 希釈の機序は本図の射程外である.

point read/write に限った観測トレース上の直列化可能性までで (述語・ファントム・公平性・未観測の実行は対象外, 3.3 節), 実行引数の記録・トレース処理の除去の証拠・条件関門の証拠・ソースの識別について 5 つの限定が付く (6 節). 正しさの記録と status は独立であり, 第 2 の走行は 2 cell とも certified でありながら reject である. certified は性能の認証ではなく, reject は正しさの証拠

を取り消さない.

なお, 同じ protocol の以前の attempt は, 静的な待機量を要求しながらパッチの無い stock のソースをビルドしており, 実際に比べたのは CCBench 内蔵の適応 backoff の有効/無効だった. その reject は当時の事実として残るが, 支持する命題は内蔵 backoff の有効/無効であり, 静的候補の失敗には用いない.

表 2 下段は別の測定で、同一候補 $5\mu\text{s}$ を 3 ワークロードで同時期に (各ワークロードで無 backoff 側の直後に採用側を) 測った。結果を見る前に、対差が $-$ 床値 (無 backoff 側の走行間の変動係数) を下回れば退行とする規則を固定した。read-heavy だけが床値を超える退行で、write-heavy と balanced は退行なしだった。論理積による status の reject は read-heavy の対差が負であることの帰結で、全ワークロードの退行を意味しない。退行した候補を含む 6 cell とも別走行で certified である。この測定は単一 attempt・記述的・非認証で、反復間の安定性は未判定である。床値判定は有意差判定ではなく、「退行なし」は優越でも差が無いことの証明でもない。バイナリはワークロードごとに別ビルドである。上段とはプールせず、上下段の同じワークロードの値も採用値・attempt・日付・ノードが異なり、比べる対差ではない。退行の機序は述べない。

4.4 事前登録した対比較の非認証 lane

旧環境の利得に対応する対比較を、事前登録した配置と推定対象の下で測った。標本設計 (各ワークロード 30 対) を結果より前に固定し、ワークロードごとに別 job の独立した campaign として、均衡した 5 反復ブロック交互の配置で無 backoff 側と採用側 ($10 / 5 / 2\mu\text{s}$) を測った。30 対の差 (採用側 $-$ 無 backoff 側) の平均の登録済み区間 (平均 $\pm h$) が床 $\pm B$ (B は無 backoff 側平均の 3%) の外なら resolved-above-floor と分類する。これは事前登録した規則の記述的な札で、 h の係数 k は arm 失敗確率の目標 $1/120$ から固定した t 分位点であって有意水準ではなく、向きは平均の符号で読む。有意差や p 値の言葉へは翻訳しない。この lane は正式な結果へ昇格させないと事前に定めた非認証の経路 (formal=false, promotion_prohibited=true) で、その反転を文書の編集で行うことは事前登録と policy が禁じている。

attempt-0001 (2026-09-18) は完走し、3 ワークロードとも resolved-above-floor を出した (対差平均 write-heavy $+1,591,948.5$, balanced $+448,830.17$, read-heavy $-576,749.77$ tps, 図 2)。差の標本標準偏差が計画値を超えたかを示す variance_plan_breach は 3 本とも false だった。attempt-0002 は同じ policy・乱数の種・物理順での独立の観測として 1 回に限り認可され、2026-09-20 に投入・完走し、3 ワークロードとも resolved-above-floor を出した (対差平均 $+1,538,451.47$, $+548,138.23$, $-560,565.60$ tps, 図 3)。標本標準偏差は write-heavy と read-heavy で計画値を超え ($80,148.44 > 66,403.45$, $80,752.99 > 74,668.49$ tps), balanced では超えていない ($49,427.36 < 56,697.44$)。分類は $|\text{平均}| - h > B$ で決まるので、超過しても変わらない。超過の原因 (ノード・時刻帯・bench 相の順序・無 backoff 側の変動) は帰属しない。

2 つの attempt はノード・時刻帯・bench 相の順序など

も異なり、どの差がどの値に影響したかは帰属しない。本稿は 2 つの観測を並べるだけで、差・比・合成区間・区間の重なり・再現判定を作らない (attempt 間の比較の事前登録は無い)。言えるのは、同一配置の反復で分類 (6 cell とも resolved-above-floor) と符号 (正・正・負) が一致した観察までである。単一 attempt を反復間の安定性へ一般化しないという限定は解除せず、2 attempt の観察に基づく限定として保つ。3 回目の attempt は行っていない。

符号は旧環境の 3 値や 4.3 節の 3 走行と同じ向きだが、推定対象 (対差の平均と中央値の比)・環境・分母が異なる観察で、再現の判定ではなく、プールもしない。両 attempt とも 6 arm が別のトレース有効走行 (legacy 構成) で certified (観測トレース上の直列化可能性に限る) だが、性能の認証ではない。この lane の出力は非認証の記述的出力であり、この実験の充足ではなく、正式化も判定していない。

4.5 合成軸と既知軸最良の直接比較

abort の要因ごとに backoff の発火可否を切り替える trigger-gating 軸の構成 (系側 gate 構成) を、事前に凍結した既知軸の最良 3 種 (コンパイル時フラグ最適化、静的 backoff の最良値、書込ロック順の並べ替え) と 3 workload で直接比較した (図 4)。これは既知の結果を見た後に登録した追試であり、凍結資料に記録された本走は旧環境 (linux-baremetal, perf stat 下) での 2026-07-16 の 1 回である。各セルは 8 標本 (2 ブロック $\times$ 4)、判定境界は相対中央値差 $+3\%$ と 2 ブロックの方向一致で、族の判定は 9 対の連言である。

書込ロック順の並べ替えに対する 3 対は $+55.5\% \sim +98.4\%$ で境界を超えたが、コンパイル時フラグ最適化には $-9.3\% \sim -55.1\%$ 、静的 backoff の最良値には $-36.0\% \sim -51.9\%$ で届かず、族の p 値は 1.0 となり、既知軸最良を超えるという事前登録した主張は不成立である。負けた 6 対はいずれも 2 ブロックで方向が一致していた。正しさは別ビルドの検査 324 件がすべて certified (観測トレース上の判定で、範囲は 3.3 節のとおり)・anomaly 0 だったが、これは性能の認証ではなく、正しさの判定と性能の判定は独立である。

同じ報告の別の族である軸の on/off 効果 (gate 述語を恒真にした対照との 3 対) は族の p 値 0.000204 で成立し、4 検定の Holm 手順 (族 α 0.05) の第 1 段 (α 0.0125) を通過した。ただしこの効果は先行する機械 sweep で既知であり、結果既知の事前登録付き追試であって新しい知見とは数えず、既知軸最良への優越も支持しない。軸提案の適格率の比較 (本アーム 20/20 に対し、選定だけを無作為化した対照 17/20, 名目 $p = 0.115$ 。生成過程全体を置き換えた対照ではない) は Holm 手順の第 2 段で有意とならず、診断数値を除いた対照 (20/20, 名目 $p = 1.0$) でも帰属遮

表 2 現行環境 (Pegasus) の対測定. 効果は無 backoff 側 (BACK.OFF=0, BACKOFF.FIXED=-1, protocol 上の stock 側 cell) に対する中央値の比で, スループットの列は中央値 (tps), 各 cell はトレース無効の 5 標本. 上段は protocol・事前登録・attempt が異なる 3 つの独立した走行であり, 1 つの実験として集計しない. status は protocol の出力で, 効果量に信頼区間も有意差判定も無い. 下段は同一候補 fixed 5 μ s の 3 ワークロード同時測定 (単一 attempt・記述的・非認証・反復間の安定性は未判定) で, 上段とプールも比較もしない. 下段の判定は結果前に固定した規則 (対差 < - 床値) による記述的な判定で, 有意差判定ではない. 絶対スループットは headline 値にしない.

ワークロード	採用側	無 backoff 側 (tps)	採用側 (tps)	効果	status / 判定
上段: 3 つの独立した走行					
第 1 の走行 (write-heavy と balanced の論理積)					
write-heavy	fixed 10 μ s	2,438,295	3,987,794	+63.5485%	observed-positive (論理積)
balanced	fixed 5 μ s	3,756,230	4,297,929	+14.4213%	(同上)
第 2 の走行					
read-heavy	fixed 2 μ s	10,088,796	9,505,248	-5.7841%	reject
第 3 の走行 (別の事前登録)					
balanced	fixed 5 μ s	3,893,509	4,330,570	+11.225375361916456%	accepted
下段: 同一候補 fixed 5 μ s の 3 ワークロード同時測定 (単一 attempt・記述的・非認証・反復間の安定性は未判定)					
write-heavy	fixed 5 μ s	2,354,846	3,953,710	+67.8968%	退行なし (- 床値 -0.9536%)
balanced	fixed 5 μ s	3,832,768	4,318,443	+12.6717%	退行なし (- 床値 -0.7250%)
read-heavy	fixed 5 μ s	10,334,945	9,158,963	-11.3787%	退行 (床値超) (- 床値 -0.2228%)

断による退化は示されなかった. 登録した確定文言のとおり, 適格率次元の発見再現性は未実証のままであり, 軸の効果の成立はこの限界を解消せず, 性能次元でも既知軸最良の超越は実証されていない.

この不成立を, 合成に価値が無いことと読まない. 既知軸集合は単軸最良を集めた凍結有限集合, すなわち合成未探索の下界であり, 言えるのは trigger-gating 1 本の構成がそれを 3 workload で 2 軸について下回ったことまでで, 6 対の差が合成で埋まりうる規模かは判定しない. 値は旧環境・旧版の測定であり, 絶対スループットを主要な結果の値に用いず, 4.3 節の現行環境の走行とも比較しない.

4.6 正しさ側の追加検証

性能とは別に, トレース有効ビルドの正しさ専用走で 2 つの追加検証を行った. いずれも性能値を含まず, 対象と長さが違うので互いに比較もプールもしない.

第 1 は, 採用した静的 backoff の 2 候補 (固定 5 μ s と 10 μ s) の検証相である. 各候補で独立 8 反復 $\times$ 3 workload (extime 3 s) の本走 24 件と, 校正で完走した 6 件を合わせた判定集合 30 件がすべて anomaly 0 で, 判定は pass だった. 校正の extime 10 s の 2 件/候補は判定器が完走せず, 判定を持たない (開示のみ). 言えるのは判定集合 30 件で anomaly 0 という操作的事実までであり, 確率の主張ではない. 最終候補の検証には数えず, 既存の certified の記録を昇格も降格もしない.

第 2 は, 4.5 節の最終候補 (系側 gate 構成) の長時間の正しさ検証である. 結果を見る前に発効させた事前登録の規則に従い, 校正 (各 workload で extime 6 s と 10 s を 1

回ずつ) から規則どおり extime を 10 s に定め, 本走は独立 8 反復 $\times$ 3 workload の 24 枠とした. 本走 24 枠と校正の完走 6 枠からなる判定集合 30 枠のすべてで anomaly 0 であり, 3 値判定は pass だった. pass は規則を機械的に適用した出力であって, 研究の成功宣告ではない. 反復ごとに種を変えることは, 各反復を独立プロセスとして起動し, 各スレッドが自己シードするという操作的定義による. シード値は記録しておらず, 乱数列の独立性そのものは確かめていない. 結果は certified の範囲 (3.3 節) を超えず, 1 回の cohort であり, 別の日・別のノード集合での再現は取っていない. 別の workload・スレッド数・extime・版へ広げず, 長さ反復数の検出力も主張しない. 扱う失敗条件は合成が certified を破るかどうかの 1 つだけで, 直接比較の事前登録が付属させた要件の充足でもない. 現行の版のソースについての検証であり, 直接比較当時のソースやバイナリの再検証ではなく, 4.5 節の不成立を変えない. 検証相の校正で 10 s が未完走だったこととも比較しない.

4.7 合成ループの到達と還流

正しさゲートを毎反復回す合成ループは, 限定したスコープで, 設定したゲートを一度も緩めずに, certified (3.3 節の範囲) の終端判定まで 4 度到達した (既存提案の再評価 1 回と手動ループ 3 巡, その後の 2 job は後述). 1 巡は, 提案役が型付き入力から値を 1 つ作り, 既存の経路が検証用・性能用の別ビルドで 1 回評価して記録し, critic が記録を読んで次の方向を書く流れである. 前の巡の実測が次の巡の入力に入る還流は 2 回, critic の診断が入る還流は 1 回成立した. ただし動かしたのは固定 backoff の初

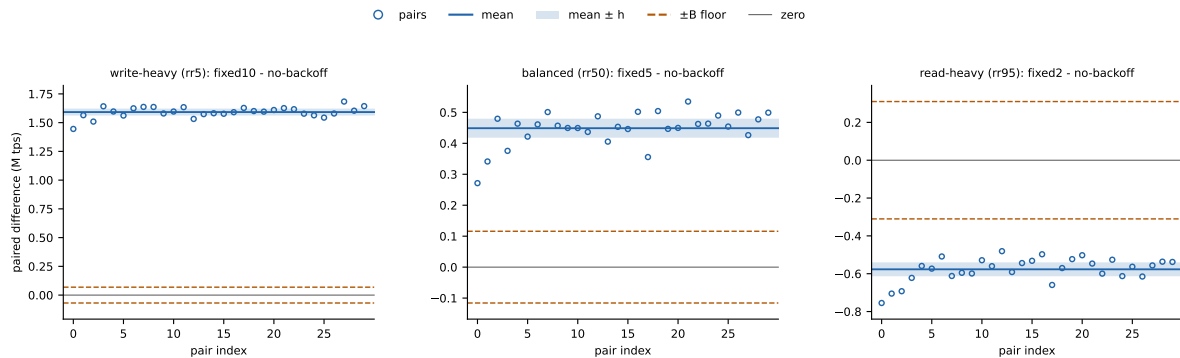


図2 事前登録した対比較の非認証 lane, attempt-0001 (formal=false, promotion_prohibited=true, 結果の権限は事前登録どおりの記述に限る). 列は write-heavy (fixed 10 μ s – 無 backoff), balanced (fixed 5 μ s – 無 backoff), read-heavy (fixed 2 μ s – 無 backoff) で, 各々別 job の独立した campaign である. 白抜き印は均衡 5 反復配置の下での 30 対の差 (採用側 – 無 backoff 側), 実線はその算術平均, 帯は登録済み区間 (平均 $\pm h$, $h = ks/\sqrt{n}$, $k = 2.8315526875186725$ は自由度 29 の t 分布の $1 - (1/120)/2$ 分位点, s は 30 対の差の標本標準偏差), 破線は登録済みの床 $\pm B$ (B は無 backoff 側平均の 3%) である. 値は write-heavy 平均 +1.592 M tps (h 0.024 M, B 0.069 M), balanced +0.449 M tps (h 0.028 M, B 0.116 M), read-heavy -0.577 M tps (h 0.033 M, B 0.310 M). 分類は 3 ワークロードとも resolved-above-floor (符号は正・正・負), variance_plan_breach は 3 つとも false. 区間と分類は事前登録した規則の記述の出力であり, 仮説検定でも性能の認証でもない. 単一 attempt の記録であり, headline 値・ワークロード横断の結論・旧環境の結果の再現判定にせず, 反復間の安定性へ一般化しない. 条件は Pegasus 計算ノード, 48 スレッド, 1,000,000 レコード, Zipf 0.9, read-modify-write 無効, max operations 10, 各反復 3 秒, ワークロードごとに 30 対, perf 無し, 性能はトレース無効. 正しさはトレース有効の別走行 (legacy 検証構成) から来て, 6 arm とも certified (観測した YCSB point read/write トレースの直列化可能性に限る) だが, 性能の認証ではない. 縦軸はパネルごとの尺度で, パネル間で比べない. pilot の観測値は推定に入っておらず, 推定対象は均衡 5 反復配置の下での差であって持ち越しの無い定常効果ではない.

期値のリテラル 1 つで, CC の構造を合成したものではない. 人間が各巡を運ぶ経路であり, ループが自律的に回ることの一般的な証拠でもない. 3 評価とも anomaly 0 で, 規律が求める「なぜ壊れたか」の反例の還流は一度も発火していない.

第 3 巡では診断が提案役の入力に届き, 提案値は診断の候補値と一致した. しかし「届いた」と「効いた」(提案値を変えた因果) ことは別であり, 各条件 1 回の起動で無作為化も反復も無いため因果は言えない. 3 巡の値は別日・別ノードの非同時刻の値であり, 性能の改善・退行の根拠にしない. 同一 job の stock 対照は 3 巡のどの巡にも無かった. 候補と stock を 1 job で評価する pair は初投入で不成立に終わった (stock 側がビルドに到達しなかった).

その後, 実行器を修復して pair を 1 job 再投入し, 初めて成立した. 第 3 巡の候補 (初期値のリテラル 10) と stock がともに certified (anomaly 0) で, stock 側が既定実装のソースからビルドされたことも照合した. 続けて, 第 3 巡の記録から組んだ入力力で提案役を 1 回ずつ起動し

た 4 巡目 (候補の値 5, 同一 job の stock 付き) も, 候補と stock がともに certified だった. ここでの stock は CCBench 既定の適応 backoff であり, 4.3 節の無 backoff 対照とは別の構成である. 同一 job 内の候補と stock のスループット比は 2.366 と 2.493 だったが, これは小さな構成 (4 スレッド, 100,000 レコード, read 比率 50%, Zipf 0.9, 各 1 秒 $\times$ 2 反復) の 2 点の記述的な値で, 性能の改善の主張ではなく, 他の構成へ広げない. 別 job の 2 候補どうしは比べない. 4 巡目の提案値は第 3 巡の critic が挙げた候補値と同じだが, これも届いたことであって効いたことではない. 4 巡目の結果を critic が読んで次の方向を書く段と記録の取り込みは本稿の時点で行っておらず, 4 巡目が 1 巡として閉じたとは言わない. 3 巡の値と判定はこの 2 job によって変わらない.

反例の還流の効果を問う on/off 比較 (構造化された拒否の詳細の有無だけを変える設計) は, 適格な赤の先行候補 (precursor) が 0 件のため実施できていない. 提案・走行・参照点の対応を記録する仕組みは実装したが, 本走は投入

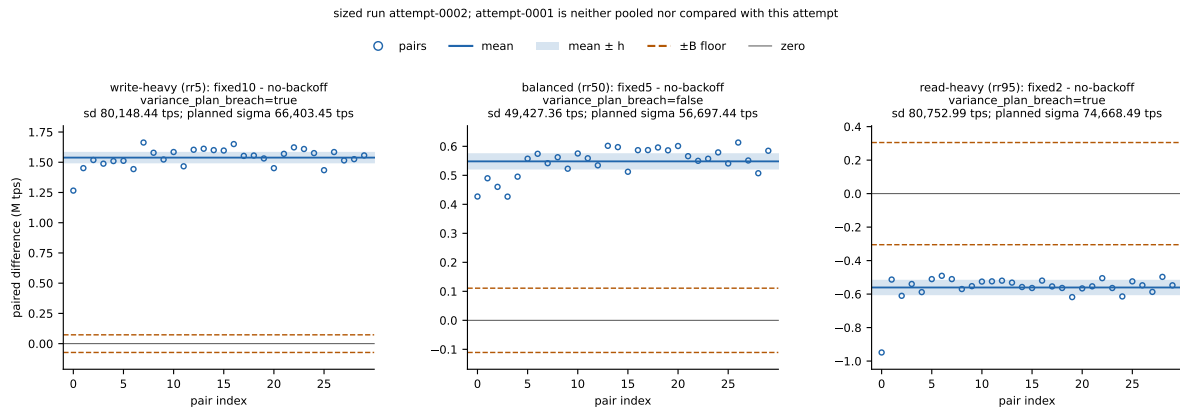


図3 事前登録した対比較の非認証 lane, attempt-0002 (formal=false, promotion_prohibited=true). attempt-0001 と同じ policy・同じ乱数の種・同じ物理順での独立の観測で、列・描画要素・測定条件・正しさの出所と限定は図2と同じである。値は write-heavy 平均 +1.538 M tps (h 0.041 M, B 0.073 M), balanced +0.548 M tps (h 0.026 M, B 0.111 M), read-heavy -0.561 M tps (h 0.042 M, B 0.305 M). 分類は3ワークロードとも resolved-above-floor (符号は正・正・負). variance_plan_breach は write-heavy (標本標準偏差 80,148.44 tps, 計画 66,403.45 tps) と read-heavy (80,752.99 / 74,668.49 tps) で true, balanced (49,427.36 / 56,697.44 tps) で false であり、その原因は帰属しない。attempt-0001 とはプールも比較もせず、2 attempt の差・比・再現の判定を作らない。区間と分類は事前登録した規則の記述的出力であり、仮説検定でも性能の認証でもない。単一 attempt の記録であり、反復間の安定性へ一般化しない。

していない。したがって還流の改善効果は未実証である。

なお3巡の実行記録の原本は記録後に作業領域の撤去で失われ、一部は派生物から sha256 が記録値と一致する複製として再構成できるが、第3巡の WAL は内容の同一まで、第3巡の critic が読んだ要約と lock は記録された sha256 までは確かめられず、材料レポートの作り直しも再実行できないため、本稿は3巡の値と判定を記録からの転記として保ち、変えない。

4.8 backoff 機序の帯域外

静的 backoff 軸の性能地形が主要な動作点の外でどう振る舞うかについて、記述的な材料を2つ得た。いずれも性能は未認証で、候補採用の根拠にせず、機序も判定しない。第1に待ち方の grid では、事前登録した一定待機と対称な剰余待機の1対比 (指示値 $\mu = 2 \sim 100 \mu\text{s}$ の6点 $\times$ 3ブロック, 48スレッド) を3族 Holm の exact 符号反転 permutation で検定し、3族とも different (剰余待機が高い側) となった。効果量の95%区間は36セル中32が等価域 $\pm 3.0\%$ の内側 (うち18は一定待機の参照セルで区間 $[0, 0]$), 4が境界を跨ぎ、完全に外側のセルは0だったが、等価性検定はしていない。第2に静的待機の右 tail ($1000 \sim 9999 \mu\text{s}$ の8点) では、この事前登録の述語では、表現可能域である9999マイクロ秒までに飽和を観測しなかった。9999 μs は符号化の上限であって物理的な

限界ではない。同じ格子で throughput は1000から9999 μs へ3 workload とも半分以上に下がり (第1 cohort の write-heavy で5反復の算術平均 993,106.4 から 401,697.6 tps), abort 率が下がり続けることと、その帯で性能が大きく失われることが同時に成り立つ。この結末は独立な2 cohort のそれぞれで成り立つが、第1 cohort を主とし、第2 cohort は再現欄に併記するだけで、統合判定やプール推定は作らない。正しさは性能条件と異なる小さな設定の別走行で anomaly 0 を確かめたもので、性能の認証ではない。この帯域外の検討は閉じていない。

4.9 別プロトコル (MOCC) の G2 観測

MOCC は合成対象の第2例の準備段階にあり、非 Silo の性能比較は0件である。その準備として、stock MOCC のトレース有効ビルドで判定器が G2 signal を出す条件を調べた観測を示す (図5)。これは非 certifying の観測記録であり、採用・認証・性能の根拠にしない。軽量 witness (読んだ値の来歴を trace に記録し、実際の異常と torn read を区別するための計装を軽くしたもの) の on/off と backoff の有無を組み合わせた4 arm を4ブロック $\times$ 15ラウンドで回したところ、on の2 arm は0/60, off の2 arm は1/60で、片側 Fisher 検定 (on が低い方向, 未調整) はどちらの比較も $p = 0.500$ だった。この標本と条件では on/off の率差を検出しなかったが、検出力 0.105 は設計仮定の下

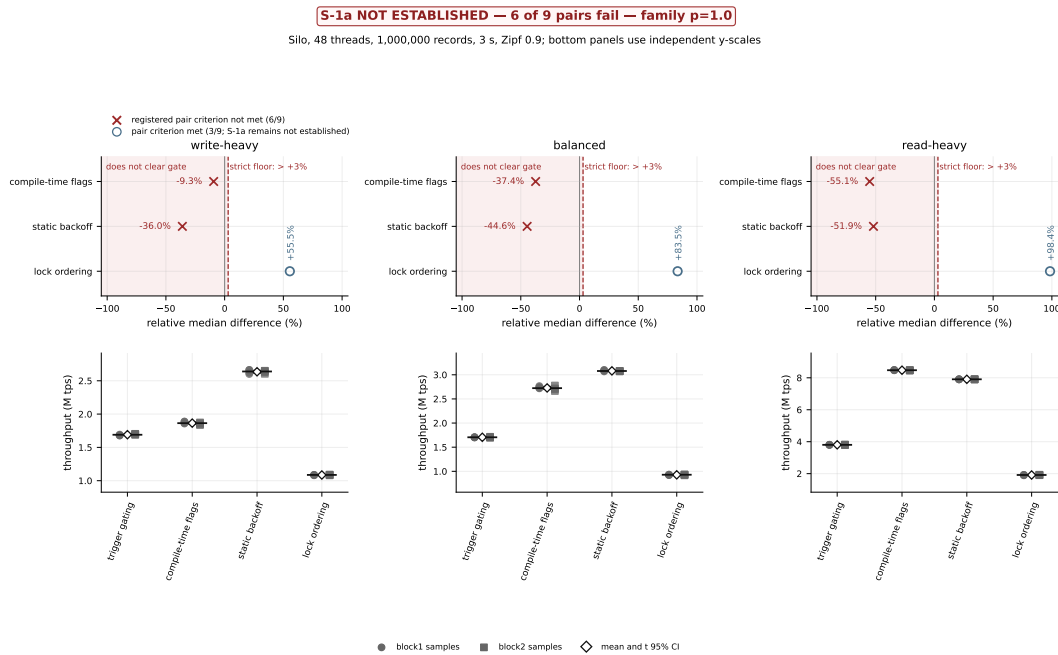


図 4 合成軸と既知軸最良の直接比較 9 対 (失敗報告図). 図中の「S-1a」は本節の直接比較の事前登録上の名前である. 上段は, abort 要因別に backoff の発火可否を切り替える合成軸 (trigger gating) と既知軸の最良 3 種 (コンパイル時フラグ最適化, 静的 backoff の最良値, 書込ロック順の並べ替え) との相対中央値差で, 各セル 8 標本から再計算した. 灰色の実線は差 0, 赤の破線は厳密に超える必要がある判定境界 +3% (走行間の再現ばらつきの下限), 薄赤の領域は境界を超えない範囲である. 成立条件は 9 対すべてが境界を超えることで, フラグ最適化との 3 対 (-9.3%~-55.1%) と静的 backoff 最良値との 3 対 (-36.0%~-51.9%) は超えず, 書込ロック順並べ替えとの 3 対 (+55.5%~+98.4%) だけが超えたため不成立である (family p = 1.0). 下段は各セルの 8 標本の全数 (各標本は同一セッション内 5 反復の中央値, M tps は毎秒 100 万トランザクション) で, 短い横線が中央値, 菱形と誤差棒が標本平均と t 分布による 95% 信頼区間である. この区間は標本分布の記述用で, 相対中央値差・判定境界・族の判定には用いていない. 下段の縦軸は workload ごとに独立で, パネル間で高さや幅を比べてはならない. 条件は旧環境 (linux-baremetal, CCBench の旧版) の Silo, 48 スレッド, 100 万レコード, Zipf 0.9, read-modify-write 無効, 3 秒, `clocks_per_us = 1,800` TSC tick/ μ s, トレース無効, `numactl --interleave=all` で, read 比率だけが workload ごとに異なる (write-heavy 5%, balanced 50%, read-heavy 95%). 測定は `perf stat` 下の記録でそのオーバーヘッドを含むため, 絶対スループットは主要な結果の値の出所ではなく, 現行の同一 campaign 内の対測定契約を満たすとも主張しない. 軸の on/off 効果の追試の成立は, 既知軸最良との優劣を問うこの不成立を救わない. 本図を候補採用の根拠にしない.

で計算した値であり, 非有意は同等性ではなく, 0 件は不在ではない. G2 signal の 2 走は判定器が非直列化可能として certified を与えず, 即時失格の契約はそのまま働いたが, signal は根因を同定しない. 図の commit 数は曝露量であって性能ではない. 先行する観測条件の分離 (別の 360 走) を含め, 過去の観測の値とは合算も比較もしない.

ロック被覆の観測点 (X/P 計装) を持たない素の固定版 (pin) の MOCC のトレースは, 判定器が indeterminate とする (SI は証明面を持たない). その後, X/P 計装を固定版の直接の子となる候補 commit として載せ, その上で正例・負例の検証走 1 本が 14 項目の検査をすべて満たし

た. ただしこの候補は固定版にまだ取り込まれておらず, MOCC の certified な系列も探索も開いていない.

5. 考察

結果が支持するのは, 既存の設定空間を探索することと, 実装が選べる操作を増やすことを分ける Izanagi の構成である. 有限フラグ空間では LLM 誘導が貪欲探索を上回らなかった (4.1 節). 一方, abort 後の待機量を静的に固定する操作を加えた後の機械的な sweep は, 旧環境の 2 負荷で無 backoff 対照 (BACK_OFF=0) を上回る点を見つけ (read-heavy では全点が下回った), 現行環境の測定 (4.3~

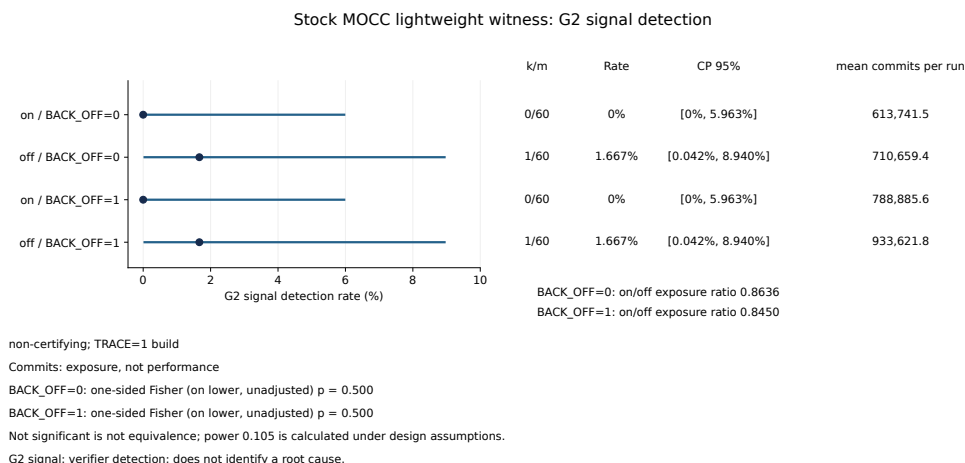


図 5 stock MOCC の軽量 witness 4 arm (各 60 走) の G2 signal 検出率 (点 = k/m, 横線 = Clopper-Pearson (CP) 両側 95% 区間) と、走あたり commit 数の平均 (記述). witness on は BACK_OFF=0 / 1 とともに 0/60 (0%, CP [0%, 5.963%]), off は両方とも 1/60 (1.667%, CP [0.042%, 8.940%]). 片側 Fisher 検定 (on が低い方向, 未調整) は BACK_OFF=0 (主比較)・1 とともに $p = 0.500$, on/off の曝露比は 0.8636 / 0.8450 である. 非有意は同等性を示さず, 0 件は不在を示さない. 検出力 0.105 は設計仮定 (独立 Bernoulli 試行, arm あたり 60 走, off の確率 0.0417, on の確率 0, α 0.05 の片側 Fisher) の下の計算値で, 実測量ではない. TRACE=1 ビルドの commit 数は曝露量であって性能ではなく, 検出率は 3 秒の固定時間の走あたりの率で, 等しい commit 曝露での比較ではない. G2 signal は根因を同定せず, 実 anomaly と torn read も区別しない (on の signal が無く識別器は一度も発火していない). 非 certifying の観測であり, 個々の判定器の certified 表示や arm の属性は, この観測・MOCC・witness を認証しない. 分母は 4 ブロック $\times$ 15 ラウンド $\times$ 4 arm だけで, smoke 走は除く. CP 区間と Fisher の p は独立な Bernoulli 試行を仮定し, ノード内相関・回転順・時間変動をモデル化していない. 過去の観測の値とは合算も比較もしない. 条件: 48 スレッド, 10,000 レコード, Zipf 0.9, read 比率 50, rmw 0, max operations 10, 3 秒, TRACE=1 ビルド, 固定版 + X/P・軽量 witness の patch, Pegasus 計算ノード.

4.4 節) も write-heavy・balanced で正, read-heavy で負という同じ向きの符号を示した. これは操作の拡張の実現可能性を支持する一事例である. ただし符号の一致は記述的な照合であって再現の判定ではなく, 走行どうしはプールしない. backoff という機構自体は既知で (Polyjuice [2] は Silo 上の backoff も学習している), この一事例は機構の新しさを示さない. LLM が生成器として必要だったことも示していない. この形の必要性は対照を取っても言えず, 固定予算での条件付き優越を問う対照も未取得である. 現行の合成空間 (backoff の数値 1 つ, 施錠順序 79 値, 発火条件 5 bit) は列挙し尽くせるので, LLM の出番も, LLM が壊した候補を判定器が捕らえる機会もほとんど無かった.

性能の改善は負荷と対照に依存した. read-heavy では旧環境 (−6.6%), 現行環境の正式な対測定 (−5.7841%), 同一候補 5 μ s の測定 (−11.3787%), 非認証 lane の 2 回の attempt (対差平均 −576,749.77 / −560,565.60 tps) がいづれも無 backoff 対照に対して負の向きで, 他の 2 負荷の正の向きと併せると「待機を増やせば一律に速くなる」と

いう解釈に反する (値はプールしない). 合成した軸は sort 軸の最良に 3 負荷とも勝ち (+55.5~+98.4%), on/off 効果も結果既知の事前登録付き追試として成立したが, 既知軸の最良は超えなかった (4.5 節). これは「軸に効果がある」と「最良の既知軸を超える」の隔たりを示すもので, 合成の価値を否定しない. したがって stock や差なしは, 証拠が改善を支持しないときの有効な出力である. ただし比較値が未取得の場合や手続きが成立していない場合は, 差なしの判定にもならない. 負荷ごとの最良点 (10 / 5 / 2 μ s) の違いだけでは, ワークロード記述を条件にした生成の因果的な効果は示されない.

abort 率とスループットの同時変化は, 競合の抑制と待機の費用の釣り合いという解釈に整合する. しかし abort 率は実験ごとに集約方法が違う記述的な先行指標で, 待ち方の比較と右裾の測定 (4.8 節) も記述的な会計と分類にとどまるので, 性能差を特定の機序へ帰属しない.

正しさと性能を別ビルド・別実行に分ける規律は実際に働いた. 以前の対測定の attempt は要求した定義マクロ

が黙って無視されたまま完走しており、正しさの合格だけでは要求した構成のビルドを保証しない事例となった (4.3 節). 逆に、退行したセルも certified (観測 trace 上の直列化可能性) であり、性能で既知軸の最良を超えなかった最終候補の長時間検証は pass だった (4.6 節). 両者は別の段の結果で、互いを補わない。

還流について言えるのは、3 巡の時点で実測値の還流 2 回と診断の還流 1 回が成立した、までである (4.7 節、その後の 4 巡目は結果を次の巡へ還流させる段が未了). 届いたことと効いたことは別で、「なぜ壊れたか」の還流は一度も発火しておらず、還流の改善効果は未実証である。

6. 限界

測定契約. 旧環境の静的 backoff の 3 値 (+38.3% / +11.3% / -6.6%) は、現行の測定契約より前に旧環境・旧版の CCBench で取った、旧測定契約下の記述的な一事例である。分母は同じ sweep 内の無 backoff 対照 (BACK_OFF=0) で、有意差判定も区間推定も無く、24 走はすべて perf stat 下なので絶対スループットは主張に使わない。現行環境の正式な対測定は 3 つの独立した走行で、信頼区間も有意差判定も持たず、旧値との符号の一致は再現の判定ではない。昇格を禁じた非認証 lane の 2 回の attempt は同一配置の反復で分類と符号が一致した観察で、正式な充足は未判定である。同一候補の同時期測定が満たしたのは退行込みの報告要件だけで、単一 attempt・記述的・非認証・反復間の安定性は未判定である。別起動での取り直しは無く、判定下限も較正していない。現行環境の走行間ばらつきの下限 (順に write-heavy / balanced / read-heavy で 0.9536% / 0.7250% / 0.2228%) は cold-boot と温度ドリフトを含まない。素材コーパスの版は測定後に前進したが、旧版での測定事実と当時の判定は無効にならない。

正しさの保証範囲. 判定器が担保するのは、YCSB の点読み・点書きに限った、観測できた trace 上の直列化可能性である。述語読み・phantom・公平性・starvation・未観測の実行は対象外で、「正しさゲートを一度も緩めなかった」も工程上の事実であって証明ではない。certified は実際にビルドされた bytes の判定で、要求した構成のビルドを含まない。採用静的 backoff の certified には、(i) point-key の trace に限る、(ii) 正しさ側の実行引数は独立に記録されていない、(iii) artifact hash 単独では compile-out の証明にならない、(iv) 測定条件の関門について残るのは受理の記録と ID まで、(v) src.token の一致は実翻訳単位全体の意味の一致を保証せず、前処理指令と include の相対位置や push_macro / pop_macro が限界として残る、の 5 限定が付き、旧環境の 3 値へは遡らない (旧 3 値の正しさは機序論証による外挿). trace 有効時は commit 数が約 44% 減り、検証が踏む競合は性能測定時の約 56% 相当と解釈している。検証相と最終候補の長時間検証の anomaly

0 (判定集合 30 件、検証相は候補ごと) は操作的な事実で、確率的な保証でも性能値でもなく、両者は対象と長さが違うので比較しない。後者は 1 回の実施で、別の workload・スレッド数・実行時間・日・計算ノード・版へ広げず、検出力も主張しない。素の版の MoCC は判定器が判定不能とし、trace 用の計装 (witness) を入れた条件で G2 が 0 件だったことは不在の証明でなく、非有意は同等性の証明でない。

合成能力. 合成軸が既知軸の最良を超えるという事前登録した主張は、旧環境で測って不成立だった。結果既知の追試として登録した複合主張の失敗報告であり、現行環境の走行はこの比較ではない。合成の一般性も、ワークロード記述を条件にした生成の因果的な証拠も無い。LLM が必要だったという形の主張は対照を取っても言えず、代わりに問う固定予算・同一編集面での条件付き優越の対照は、事前登録が未発効で本走は未認可である。合成ループが評価したのは固定 backoff の初期値で CC の構造ではなく、還流の改善効果を問う比較は適格な失敗候補が 0 件で実施できない。有限フラグ空間の否定的結果は 3 負荷のうち 2 つについてのもので、LLM によるコード生成一般の無益さは推論できない。backoff の機序として閉じているのは 0~10 μ s の帯だけで、それより長い待機での機序は同定していない (4.8 節)。

スコープ. 探索・合成が回ったのは Silo 1 protocol の backoff / sort / trigger-gating の 3 軸だけで、負荷は YCSB の 3 種 (読み比率 5 / 50 / 95%), 48 スレッドに限った (4.7 節の合成ループの評価だけは 4 スレッド・100,000 レコードの小構成). TPC-C は評価していない。合成の対象を Silo に限る方針は解除したが、第 2 の対象 (MOCC) を加えた後も狙う主張は「指定した二つの CC 実装で合成・評価手順を実証した」に限り、本稿の時点では非 Silo の性能比較は 0 件である。MoCC と TicToc は較正の記録各 2 件で性能による選定は 0 件、Cicada は測定 0 件、Snapshot Isolation は証明面を持たない。

先行研究調査. 優先権は主張せず、言えるのは 1 節の 1 文までで、系譜の中の順序も数えない。関連研究の調査は 2 軸とも費用対効果を理由に限定付きで止めており、停止は完了ではない。調査後に確認した研究と 1 文との関係は判定していない (2 節)。

7. おわりに

本稿は、対象実装の動作の集合そのものを LLM にコードとして拡張させ、生成のたびに直列化可能性を検査しながら、ワークロードごとに certified な選択を出せるかを問うた。第 1 に、そのようなループは限定したスコープで成立した。Silo の 1 プロトコルと 3 軸に限り、設定した正しさゲートを緩めずに合成を certified (観測 trace 上の直列化可能性、6 節の限定付き) まで回し、当時の版で

終端の判定へ 4 度到達した。これは運用上の証拠であり、ゲートを緩めなかったことも工程上の事実であって証明ではない。評価したのは固定 backoff の初期値で CC の構造ではなく、ワークロード条件付き合成の因果的な証拠でもない。第 2 に、拡張した動作が性能を変えることは一事例として示された。フラグ空間の外へ合成した「abort 後の待機量の静的固定」は、旧測定契約下の記述的な sweep で無 backoff 対照に対し write-heavy +38.3%, balanced +11.3%, read-heavy -6.6% を示したが、示すのは実現可能性までで、backoff の機構の新しさではない。

否定的結果が価値の所在を決めた。有限フラグ空間では LLM 誘導探索が貪欲探索より少ない評価回数で最良群に到達するという優越は示されず、欺瞞的な負荷では誘導側の到達コストが高かった。これと空間外合成の一事例を対にすると、価値はフラグ空間の探索ではなく空間の外への合成にあるという対比が得られる。一方、合成軸が既知軸の最良を超えるという事前登録した主張は不成立で（結果既知の追試の失敗報告）、本稿の主張は一事例の実現可能性へ狭まった。未実証のまま残るのは、既知軸最良の超越、ワークロード記述を条件とする生成の因果効果、正しさシグナルの還流による改善、生成器の条件付き優越、backoff の機序の同定、非 Silo プロトコルでの選定、現行契約下の対測定の正式な充足である。最終候補の長時間検証の pass は正しさ側の結果で、これらのどれも埋めない。

今後の課題を 4 つ挙げる。(a) と (b) は実装の途中で、TPC-C の認定と、LLM による方策の生成・評価は行っていない。(c) は主標本に含めない上限付きの試走だけが完走し、本走は未認可である。(d) は同一 job の対照が取れたが、4 巡目の還流は閉じていない。(a) TPC-C を trace の判定器で直列化可能性まで認定すること。段 1 は範囲読みを使わない NewOrder / Payment を表の識別子を加えた trace で扱い、段 2 は全 5 取引へ広げて範囲読みを述語読みとして扱い、phantom の依存を検出する。段 1 を先に閉じてから段 2 へ広げる順で進めると決めた。段 1 は判定器が表を区別して trace を読む側と、CCBench の Silo が表と取引種別を trace に出す側を実装し、trace の構造を確かめる走行を 1 回行った。ただし取引が読んだ行の存在履歴の検査が未実装の間は TPC-C の trace を certified と判定せず、CCBench 側の変更は素材コーパスの固定版に取り込んでいない。(b) LLM が関数単位で方策のコードを書く合成空間。Silo の abort 後の待機量、lock 競合時の再試行か中断か、commit 時の状態更新の 3 関数（と補助関数・状態型）だけを開き、受理する C++ を型付きの部分言語に閉じる。条件付きで採用した設計で、手書きの方策が受理検査・ビルド・検証・性能測定の経路を通ることまでを確かめたが、LLM による方策の生成と評価はまだ行っていない。検証と lock の仕組みは開かないため、LLM が壊した CC の論理を判定器が捕らえることの実証

にはならない。(c) 同一予算・同一編集面で非 LLM 生成器と比べる対照。上限付きの試走（1 系列ずつ、主標本外）は完走したが優劣も同等性も言えず、共通の前段検査は実装したが、事前登録は未発効で本走は未認可である。(d) 合成ループの同一 job 内の stock 対照。初回は対を投入したが不成立で、実行器を修復した後の再投入で成立し、続く 4 巡目も同一 job の stock 対照付きで評価した（4.7 節）。4 巡目の結果を次の巡へ還流させる段は未了である。

参考文献

- [1] Tanabe, T., Hoshino, T., Kawashima, H. and Tatebe, O.: An Analysis of Concurrency Control Protocols for In-Memory Databases with CCBench, *Proc. VLDB Endow.*, Vol.13, No.13, pp.3531–3544 (2020).
- [2] Wang, J., Ding, D., Wang, H., Christensen, C., Wang, Z., Chen, H. and Li, J.: Polyjuice: High-Performance Transactions via Learned Concurrency Control, *Proc. OSDI*, (2021). arXiv:2105.10329.
- [3] Pan, H., Cai, S., Dinh, T.T.A. et al.: Modeling Concurrency Control as a Learnable Function, *Proc. ACM Manag. Data*, Vol.4, No.3 (2026). DOI:10.1145/3802088, arXiv:2503.10036.
- [4] Zhou, W., Wang, Z., Peng, Z. et al.: ATCC: Adaptive Concurrency Control for Unforeseen Agentic Transactions, arXiv:2603.13906 (2026).
- [5] Raza, A., Nicholson, H., Tsakalidou, I. et al.: Declarative Concurrent Data Structures, arXiv:2404.13359 (2024).
- [6] Van Aken, D., Pavlo, A., Gordon, G.J. and Zhang, B.: Automatic Database Management System Tuning Through Large-scale Machine Learning, *Proc. SIGMOD*, pp.1009–1024 (2017).
- [7] Zhang, J., Liu, Y., Zhou, K. et al.: An End-to-End Automatic Cloud Database Tuning System Using Deep Reinforcement Learning, *Proc. SIGMOD*, pp.415–432 (2019).
- [8] Li, G., Zhou, X., Li, S. and Gao, B.: QTune: A Query-Aware Database Tuning System with Deep Reinforcement Learning, *Proc. VLDB Endow.*, Vol.12, No.12, pp.2118–2130 (2019).
- [9] Wang, J., Trummer, I. and Basu, D.: UDO: Universal Database Optimization Using Reinforcement Learning, *Proc. VLDB Endow.*, Vol.14, No.13, pp.3402–3414 (2021).
- [10] Trummer, I.: DB-BERT: A Database Tuning Tool that “Reads the Manual”, *Proc. SIGMOD*, pp.190–203 (2022).
- [11] Lao, J., Wang, Y., Li, Y. et al.: GPTuner: A Manual-Reading Database Tuning System via GPT-Guided Bayesian Optimization, *Proc. VLDB Endow.*, Vol.17, No.8, pp.1939–1952 (2024).
- [12] Zhang, X., Chen, T., Han, Z. et al.: Why Database Manuals Are Not Enough: Efficient and Reliable Configuration Tuning for DBMSs via Code-Driven LLM Agents, *Proc. VLDB Endow.*, Vol.19, No.6, pp.1358–1371 (2026). arXiv:2603.22708.
- [13] Kraska, T., Beutel, A., Chi, E.H., Dean, J. and Polyzotis, N.: The Case for Learned Index Structures, *Proc. SIGMOD*, pp.489–504 (2018).
- [14] Ding, J., Minhas, U.F., Yu, J. et al.: ALEX: An Updatable Adaptive Learned Index, *Proc. SIGMOD*, pp.969–984 (2020).
- [15] Ferragina, P. and Vinciguerra, G.: The PGM-index: A

- Fully-Dynamic Compressed Learned Index with Provable Worst-Case Bounds, *Proc. VLDB Endow.*, Vol.13, No.8, pp.1162–1175 (2020).
- [16] Marcus, R., Negi, P., Mao, H. et al.: Neo: A Learned Query Optimizer, *Proc. VLDB Endow.*, Vol.12, No.11, pp.1705–1718 (2019).
- [17] Marcus, R., Negi, P., Mao, H. et al.: Bao: Making Learned Query Optimization Practical, *Proc. SIGMOD*, pp.1275–1288 (2021).
- [18] Romera-Paredes, B., Barekatin, M., Novikov, A. et al.: Mathematical Discoveries from Program Search with Large Language Models, *Nature*, Vol.625, No.7995, pp.468–475 (2024).
- [19] Novikov, A., Vū, N., Eisenberger, M. et al.: AlphaEvolue: A Coding Agent for Scientific and Algorithmic Discovery, arXiv:2506.13131 (2025).
- [20] Lange, R.T., Imajuku, Y. and Cetin, E.: ShinkaEvolue: Towards Open-Ended and Sample-Efficient Program Evolution, arXiv:2509.19349 (2025).
- [21] Cheng, A., Liu, S., Pan, M. et al.: Barbarians at the Gate: How AI is Upending Systems Research, arXiv:2510.06189 (2025).
- [22] Liu, S., Krentsel, A., Agarwal, S. et al.: The Time is Here for Just-in-Time Systems: Challenges and Opportunities, arXiv:2605.24096 (2026).
- [23] Kamahori, K., Li, S., Peter, S. and Kasikci, B.: VibeServe: Can AI Agents Build Bespoke LLM Serving Systems?, arXiv:2605.06068 (2026).
- [24] Agarwal, S., Krentsel, A., Liu, S. et al.: Inductive Deductive Synthesis: Enabling AI to Generate Formally Verified Systems, arXiv:2605.23109 (2026).
- [25] Zhang, K. and Liu, G.: CIR+CVN: Bridging LLM Semantic Understanding and Petri-Net Verification for Concurrent Programs, arXiv:2604.09318 (2026).
- [26] Yang, Y., Gong, Z., Huang, W. et al.: SkilOpt: Executive Strategy for Self-Evolving Agent Skills, arXiv:2605.23904 (2026).
- [27] Zhang, H., Zhang, S., Li, K. et al.: Self-Harness: Harnesses That Improve Themselves, arXiv:2606.09498 (2026).